How Large Language Models Work

From Tokens to Transformers

1. The Basic Idea Behind an LLM

At the most fundamental level, a Large Language Model performs a surprisingly simple task:

> **Given everything written so far, predict what should come next.**

Consider:

I drink coffee every ____

Possible completions could be:

- morning
- day
- evening

Now add more context:

I drink coffee every morning before going to ____

The likely answers change:

- work
- office
- college
- gym

Add even more information:

I work remotely as a software engineer. Every morning I drink coffee before going to my ____

Now **desk** becomes a very likely answer.

The important idea is that the next piece of text is not decided only by the immediately previous word. It depends on the **entire available context**.

---

2. LLMs Predict the Next Token

Suppose the input is:

```
The capital of India is
```

The model may internally produce something conceptually similar to:

Possible Next Token	Probability
Delhi	94%
Punjab	2%
Bangalore	1%
Hyderabad	1%
Others	2%

These numbers are only illustrative.

The general process is:

```
Previous Text
      ↓
    Model
      ↓
Possible Next Tokens
      ↓
Probabilities
```

One token is selected and appended to the existing sequence.

```
The capital of India is
        ↓
      Delhi
```

The new context becomes:

```
The capital of India is Delhi
```

The model then predicts the token that should come after **Delhi**.

This process repeats continuously.

First Mental Model

```
Predict one piece
      ↓
Append it
      ↓
Predict another piece
      ↓
Repeat
```

There is one important correction, however:

> An LLM does not technically predict the next **word**.
>
> It predicts the next **token**.

---

3. Why Text Must Become Numbers

Computers ultimately perform mathematical operations.

A neural network works with operations such as:

- addition
- multiplication
- matrix operations
- weighted combinations

Therefore, text such as:

```
I love programming
```

must eventually be converted into numerical representations.

Imagine a very small vocabulary:

```
cat
dog
apple
car
```

We could assign IDs:

```
cat   → 1
dog   → 2
apple → 3
car   → 4
```

This allows the computer to identify each item.

But these numbers do **not** represent meaning.

For example:

```
car = 4
cat = 1
```

does not mean:

```
car > cat
```

or that a car is somehow four times a cat.

The numbers are simply identifiers.

Think of student roll numbers:

```
Rohit  → 104
Aditya → 237
```

237 being greater than 104 does not mean Aditya is “more student” than Rohit.

Important

> **Token ID ≠ Meaning of the token**

A token ID only tells the system **which token** we are referring to.

---

4. What Exactly Is a Token?

A common beginner assumption is:

> One word = one token

That is not necessarily true.

For example, depending on the tokenizer:

```
Join Coder Army Membership
```

might become several tokens.

Similarly, a word such as:

```
Elephants
```

may potentially be represented using more than one token.

A token can represent:

- a complete word
- part of a word
- punctuation

- a symbol
- a space-related unit
- another frequently occurring text fragment

Therefore:

> **One token is NOT always one word.**

Another common statement is:

> 1 token ≈ 4 characters

This can sometimes be useful as a rough estimate for English text, but it is **not the definition of a token**.

---

5. Why Not Simply Use Words?

Suppose every complete word received one vocabulary entry.

For example:

```
Automatically → Token 43
```

Now consider:

```
Automatically
Automatic
Automate
Automation
```

If every possible word had its own token, the vocabulary could become extremely large.

Languages also continuously create:

- new terms
- names

- abbreviations
- technical words
- variations of existing words

Using only whole words would therefore be inefficient.

---

6. Why Not Use Individual Characters?

We could go to the opposite extreme and tokenize every character.

```
programming
```

would become:

```
p r o g r a m m i n g
```

This avoids maintaining an enormous word vocabulary.

However, sequences become much longer.

Modern tokenizers therefore usually work somewhere between:

```
Individual Characters ← Tokens → Whole Words
```

For example, conceptually:

```
unbelievable
```

could become:

```
un + believ + able
```

The actual split depends entirely on the tokenizer being used.

Different models may use different tokenizers.

Therefore:

```
Same Sentence
+
Different Tokenizer
=
Potentially Different Tokens
```

Common sequences may often be represented efficiently, while unusual text may require more token fragments.

This is also why API providers measure usage in **tokens rather than words**.

---

7. From Text to Token IDs

Consider a simplified tokenizer vocabulary:

Token	ID
I	51
love	872
programming	4381
coffee	992

The process becomes:

```
"I love programming"
          ↓
["I", "love", "programming"]
          ↓
[51, 872, 4381]
```

We therefore have three different things:

1. **Original Text**
2. **Tokens**
3. **Token IDs**

During generation, the reverse mapping eventually allows the predicted token ID to be turned back into readable text.

For example:

```
4381
 ↓
programming
```

---

8. Why Word Order Matters

Consider:

```
Dog bites man.
```

and:

```
Man bites dog.
```

The main words are almost identical.

But the meaning is completely different.

Similarly:

```
Rohit teaches Aditya.
```

is very different from:

```
Aditya teaches Rohit.
```

The model therefore cannot treat language as a simple unordered collection:

```
{Rohit, teaches, Aditya}
```

It needs information about **where each token appears**.

Conceptually:

Token	Position
The	1
dog	2
chased	3
the	4
man	5

The representation entering the Transformer therefore needs information about:

> **What the token is + where the token appears**

Conceptually:

```
Token Representation
        +
Position Information
        ↓
Position-aware Representation
```

But knowing position alone is still not sufficient.

Language also depends heavily on **context**.

---

9. Context Changes Meaning

Consider:

```
I deposited money at the bank.
```

Here, **bank** refers to a financial institution.

Now consider:

```
We sat on the bank of the river.
```

The same word now has a different meaning.

A fixed representation of “bank” cannot fully capture both situations.

Its interpretation must depend on nearby words.

For the first sentence:

```
deposited
money
```

are highly relevant.

For the second:

```
river
sat
```

become relevant.

Therefore:

> **Tokens should not be interpreted independently. Their representations need to change according to context.**

This brings us to **embeddings and attention**.

---

10. Embeddings — Giving Tokens Meaningful Numerical Representations

A token ID tells the computer **which token** it is dealing with.

But the model needs something more useful for actual computation.

Consider:

```
king
queen
banana
laptop
```

Human beings immediately associate them with different ideas.

For teaching purposes, imagine that words were represented using features such as:

Word	Royalty	Food	Living	Technology
King	0.95	0.01	0.90	0.02
Queen	0.96	0.01	0.90	0.02
Banana	0.00	0.99	0.20	0.00
Laptop	0.00	0.00	0.00	0.98

These are **not real embedding dimensions**.

Real models do not have clean human-readable dimensions such as “Royalty” or “Food”.

Instead, the model learns mathematical features automatically during training.

So rather than using:

```
king → 781
```

as the actual representation throughout processing, the token receives a vector:

```
king
 ↓
[0.41, -0.82, 1.34, 0.15, ...]
```

This vector is called an **embedding**.

The overall pipeline now becomes:

```
Text
 ↓
```

```
Tokens
 ↓
Token IDs
 ↓
Embeddings
 ↓
Neural Network Processing
```

11. Why Embeddings Alone Are Not Enough

An initial embedding provides a general representation of a token.

But the meaning needed for a particular sentence may depend on other tokens.

Take:

```
I deposited money in the bank.
```

and:

```
I sat on the bank of the river.
```

The initial embedding for **bank** may begin similarly.

But after considering the context, the internal representation should become different.

Conceptually:

```
Initial Representation of "bank"
            +
Information from Other Tokens
            ↓
Context-Specific Representation
```

This is where **attention** becomes important.

12. What Attention Does

Consider:

```
Rohit gave Aditya his laptop because his computer was broken.
```

To understand **his**, we need to examine other words in the sentence.

Human beings naturally connect relevant pieces of information instead of treating every word equally.

Attention gives a neural network a mechanism for doing something similar.

At a high level, each token is effectively trying to determine:

> **Which other tokens are important to me, and how much should they influence my representation?**

Consider:

```
The animal didn't cross the street because it was too tired.
```

For the token **it**, the word **animal** may be particularly important.

Now change one word:

```
The animal didn't cross the street because it was too wide.
```

Suddenly, **street** becomes the more sensible reference.

The relationship changes based on context.

Important

> **Attention is dynamic.**

The model does not permanently learn:

```
"it" → animal
```

The connection depends on the current sequence.

13. Query, Key and Value

The three most famous terms in attention are:

- **Query**
- **Key**
- **Value**

A useful analogy is search.

Suppose you search:

```
best Java course
```

Your search text is the **Query**.

The system compares the query with information describing different pages. Those descriptions are similar to **Keys**.

After finding relevant pages, it retrieves useful information from them. That information is similar to the **Values**.

Conceptually:

Query

```
What information am I looking for?
```

Key

```
What kind of information do I contain?
```

Value

```
What information can I contribute?
```

In an actual Transformer, these are not English questions.

They are mathematical **vectors**.

---

14. Every Token Produces Q, K and V

Consider:

```
The cat sat on the mat.
```

Every token develops three learned representations:

```
"The"
 ├── Query
 ├── Key
 └── Value

"cat"
 ├── Query
 ├── Key
 └── Value

"sat"
 ├── Query
 ├── Key
 └── Value
```

and similarly for the remaining tokens.

Why three different representations?

Because the same information may need to be viewed differently depending on what we are trying to determine.

Consider describing a person.

For:

```
Who knows Java?
```

technical skills matter.

For:

```
Who lives closest to me?
```

location matters.

For:

```
Who should teach this class?
```

we may care about:

- subject knowledge
- communication ability
- availability

It is the same person, but we examine different aspects depending on the task.

Q, K and V give attention separate learned representations for:

```
asking
matching
communicating information
```

---

15. The Core Self-Attention Process

Suppose we are processing the token:

```
it
```

The model takes:

```
Query("it")
```

and compares it with the **Keys** of other relevant tokens.

Conceptually:

```
Query("it")
      ↓
Key("animal")  → High relevance
Key("street")  → Medium relevance
Key("because") → Low relevance
```

These relevance scores are converted into weights.

The corresponding **Values** are then combined.

Conceptually:

```
New Representation of "it"

≈ 0.60 × Value(animal)
+ 0.20 × Value(street)
+ 0.05 × Value(cross)
+ ...
```

The numbers are only illustrative.

The result is a new vector for **it** containing information gathered from its context.

This mechanism is called:

Self-Attention

It is called **self-attention** because tokens from the same sequence attend to other tokens in that sequence.

---

16. What Is a Transformer?

A common misconception is:

```
Transformer = Attention
```

That is not correct.

Attention is an important **component inside a Transformer**.

A Transformer contains repeated processing blocks.

Simplified:

```
Input Representations
        ↓
Transformer Block 1
        ↓
Transformer Block 2
        ↓
Transformer Block 3
        ↓
       ...
        ↓
Transformer Block N
        ↓
Final Representations
```

GPT-style models contain many such layers.

---

17. Why Multiple Transformer Layers?

Some relationships in language are simple.

Consider:

```
The developer couldn't deploy the application because the server ran out of memory.
```

A direct relationship might be:

```
server ↔ memory
```

But a more abstract understanding is:

```
Deployment failed
because
the server lacked sufficient resources
```

The model does not necessarily develop every useful relationship in a single step.

Instead, representations are processed repeatedly.

Conceptually:

```
Layer 1
Basic relationships
        ↓
Layer 2
Richer combinations
        ↓
Layer 3
More contextual relationships
        ↓
...
        ↓
Final contextual representation
```

A useful analogy is photo editing.

You may go through:

```
Raw Image
 ↓
Exposure
 ↓
Contrast
 ↓
Colour
 ↓
Masking
 ↓
Details
 ↓
Final Image
```

Similarly, Transformer layers repeatedly refine token representations.

A Transformer block itself contains several smaller neural-network operations; it is not simply one single operation.

18. What Travels Through Transformer Layers?

After a self-attention operation, every token receives a new contextual representation.

For example:

```
Token 1 → Updated Vector
Token 2 → Updated Vector
Token 3 → Updated Vector
...
```

These representations continue through later Transformer blocks and are repeatedly transformed.

By the final layers, each token's representation contains much richer information about:

- the token itself
- its position
- surrounding context
- relevant relationships with other tokens

19. From Final Representation to the Next Token

Consider:

```
The capital of India is
```

After passing through all Transformer layers, the final position has a contextual vector.

Conceptually:

```
Hfinal = [0.4, -1.2, 0.8, ...]
```

At this stage, the question becomes:

> Out of every token in the model's vocabulary, which token should come next?

Suppose the vocabulary contains 100,000 tokens.

The model needs approximately one score for each candidate token.

Conceptually:

```
Delhi    → 12.8
Mumbai   → 8.1
Kolkata  → 5.2
Punjab   → 3.4
banana   → -4.1
...
```

These raw scores are called:

Logits

The model has not produced probabilities yet.

It has produced numerical scores representing how suitable different tokens are.

---

20. Softmax — Turning Logits Into Probabilities

The logits are converted into probabilities using a mathematical function called:

Softmax

Conceptually:

```
Logits
  ↓
Softmax
  ↓
Probability Distribution
```

For example:

Token	Probability
Delhi	92%
Kolkata	2%
Punjab	2%
Mumbai	1%
Chennai	1%
Others	2%

The probabilities together form a distribution over the model's vocabulary.

This is one of the most important ideas behind an autoregressive LLM:

> **Given the preceding tokens, generate a probability distribution for the next token.**

Fundamentally, the model itself is not directly thinking:

```
Write an essay.
```

or:

```
Explain Java.
```

At the generation level, it repeatedly performs:

```
Given Context
      ↓
Predict Next-Token Distribution
```

Complex responses emerge through repetition and what the model learned during training.

---

21. Autoregressive Generation

Suppose the prompt is:

```
The capital of India is
```

The model produces probabilities and **Delhi** is selected.

The sequence becomes:

```
The capital of India is Delhi
```

Now the model runs next-token prediction again.

It may produce:

```
"."     → 75%
","     → 8%
"and"   → 5%
"which" → 4%
...
```

Suppose `"."` is selected.

Now we have:

```
The capital of India is Delhi.
```

The process repeats again.

Therefore, the model does **not normally generate an entire answer in a single step**.

It generates incrementally:

```
Prompt
 ↓
Predict Token 1
 ↓
Append Token 1
 ↓
Predict Token 2
 ↓
Append Token 2
 ↓
Predict Token 3
 ↓
...
```

This is known as:

Autoregressive Generation

Auto

The model uses its own previously generated output as part of the next input context.

Regressive

Future predictions depend on preceding values/context.

---

22. Why ChatGPT Can Stream Text

When using an LLM interface, text may appear gradually:

```
Java is
Java is a
Java is a programming
```

```
Java is a programming language
...
```

This is possible because generation itself happens incrementally.

Applications can send generated tokens or chunks to the interface as they become available.

Two ideas should therefore be separated:

```
Generation happens incrementally.
```

and:

```
The interface can stream that generation incrementally.
```

---

23. If the Model Has Probabilities, Which Token Is Selected?

Suppose the model produces:

Token	Probability
Java	40%
Python	35%
C++	10%
JavaScript	8%
Rust	4%
Other	3%

There are different strategies for deciding what to choose.

These strategies belong to the **decoding process**.

---

24. Greedy Decoding

The simplest strategy is:

> Always choose the token with the highest probability.

For example:

```
Java   → 40% ← Choose
Python → 35%
C++    → 10%
...
```

This is called:

Greedy Decoding

It is “greedy” because at every step it chooses whatever looks best **at that particular moment**.

Limitation

Natural language often has many valid continuations.

Always selecting the most probable token can make text:

- predictable
- repetitive
- less varied

Therefore, another strategy is commonly used.

---

25. Sampling

Instead of automatically selecting the highest-probability token, we can treat the probabilities like a **weighted lottery**.

Suppose:

```
Java   → 40%
Python → 35%
C++    → 15%
Rust   → 10%
```

Imagine 100 tickets:

```
40 Java tickets
35 Python tickets
15 C++ tickets
10 Rust tickets
```

One ticket is selected.

Java is the most likely result, but it is not guaranteed.

Python could be selected.

C++ or Rust could also occasionally appear.

This is:

Sampling

The key idea is:

> **Higher probability = more likely to be chosen, but not guaranteed.**

This helps explain why the same prompt may sometimes produce different responses.

---

26. Temperature

Temperature controls how sharp or flat the model's probability distribution becomes before sampling.

Before softmax, suppose the logits are:

```
Python     → 5
Java       → 4
```

```
JavaScript → 3
C++        → 2
Rust       → 1
```

Conceptually:

```
Adjusted Logit =
Original Logit / Temperature
```

Softmax is then applied to the adjusted values.

---

27. Temperature = 1

If:

```
Temperature = 1
```

then:

```
5 / 1 = 5
4 / 1 = 4
3 / 1 = 3
2 / 1 = 2
1 / 1 = 1
```

Nothing changes.

So:

> **Temperature 1 can be treated as the baseline case.**

---

28. Low Temperature

Suppose:

```
Temperature = 0.5
```

Then:

```
5 / 0.5 = 10
4 / 0.5 = 8
3 / 0.5 = 6
2 / 0.5 = 4
1 / 0.5 = 2
```

Notice that the differences between logits become larger.

```
Before:
5, 4, 3, 2, 1

After:
10, 8, 6, 4, 2
```

After softmax, the strongest candidates become even more dominant.

Lower Temperature

Generally produces outputs that are:

- more predictable
- more consistent
- less varied

In simple terms:

> **Lower temperature sharpens the probability distribution.**

---

29. High Temperature

Suppose:

```
Temperature = 2
```

Now:

```
5 / 2 = 2.5
4 / 2 = 2
3 / 2 = 1.5
2 / 2 = 1
1 / 2 = 0.5
```

The logits move closer together.

```
Before:
5, 4, 3, 2, 1

After:
2.5, 2, 1.5, 1, 0.5
```

After softmax, lower-ranked tokens receive relatively more probability.

Therefore:

> **Higher temperature flattens the distribution and gives less-likely tokens a greater chance of being selected.**

This usually increases:

- variation
- unexpected wording
- diversity of continuations

But it can also increase the chances of:

- mistakes
- irrelevant continuations
- nonsensical output

---

30. Model vs Decoding Strategy

This distinction is extremely important.

The **model** produces something like:

```
Here is the probability distribution
for the next token.
```

The **decoder** decides:

```
Given this probability distribution,
how should a token actually be selected?
```

So these are two separate stages:

```
MODEL
 ↓
Next-Token Probability Distribution
 ↓
DECODING STRATEGY
 ↓
Selected Token
```

Strategies such as greedy decoding, sampling and temperature affect **how we choose from the model's predictions**.

Complete LLM Generation Pipeline

We can now combine the entire process.

```
User Text
    ↓
Tokenizer
    ↓
Tokens
    ↓
```

```
Token IDs
    ↓
Embeddings
    ↓
Position Information
    ↓
Transformer Layers
    ↓
Self-Attention
    ↓
Contextual Representations
    ↓
Final Representation
    ↓
Vocabulary Scores / Logits
    ↓
Softmax
    ↓
Next-Token Probabilities
    ↓
Decoding / Sampling
    ↓
Selected Token
    ↓
Append Token to Context
    ↓
Repeat
```

Summary

An LLM can be understood through a few core ideas:

1. Text is converted into tokens

Models process **tokens**, not directly words or sentences.

2. Tokens receive numerical representations

Token IDs identify tokens, while **embeddings** provide useful learned vectors.

3. Position matters

The model needs to distinguish between:

```
Dog bites man
```

and:

```
Man bites dog
```

4. Context changes meaning

The same token can mean different things depending on surrounding tokens.

5. Attention connects relevant information

Each token can gather information from other tokens based on relevance.

6. Transformers repeatedly refine representations

Multiple Transformer layers progressively produce richer contextual representations.

7. The final representation is mapped to vocabulary scores

These raw scores are called **logits**.

8. Softmax creates probabilities

Each possible next token receives a probability.

9. A decoding strategy chooses one token

The decoder may use greedy selection or sampling.

10. The chosen token is appended and the process repeats

This repeated next-token prediction is the basis of **autoregressive text generation**.

In One Line

> **An LLM takes the existing sequence of tokens, builds context-aware mathematical representations through Transformer layers, predicts a probability distribution over possible next tokens, selects one, appends it to the sequence, and repeats the process.**